Leopold-Franzens Universität Innsbruck
Faculty of Mathematics, Computer Science and Physics

Department of Computer Science

Bachelor's Thesis

submitted for the degree of

Bachelor of Science

Symbolic Regression for Surrogate Modeling - A Systematic Study

von

Tobias Albrecht
(Matr.-Nr.: 12206456)

Submission Date: TODO: Add Submission Date
Supervisor: Prof. Philipp Zech, PhD

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt durch meine eigenhändige Unterschrift, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder inhaltlich den angegebenen Quellen entnommen wurden, sind als solche kenntlich gemacht.
Ich erkläre mich mit der Archivierung der vorliegenden Bachelorarbeit einverstanden.

Ort und Datum: _____

Unterschrift: _____

Disclaimer on the Use of AI

TODO: Add Disclaimer on the Use of AI

Abstract

Surrogate models are often used in scientific and engineering applications to approximate computationally expensive simulations. However, many of the currently used surrogate models are black box models which provide almost no insight into the process of their actual learnings. Symbolic Regression (SR) addresses this limitation by discovering explicit mathematical expressions directly from the input data, therefore enabling better model interpretability while maintaining predictive performance.

This thesis provides a systematic study of Symbolic Regression methods for surrogate modeling. The main goal is to compare open-source SR frameworks that employ different algorithmic approaches, including evolutionary, genetic programming, transformation-based, and neural methods. The evaluated frameworks are gplearn, Operon, PySR, GeneticEngine, ITEA, and EQL.

To evaluate these frameworks, the benchmark usses datasets from different domains, including engineering, physics, material science and energy systems. The selected SR models are evaluated using identical training and test data and a common evaluation protocol with documented algorithm-specific search budgets. The evaluation considers predictive accuracy, computational efficiency, model complexity, stability across repeated trials, and sensitivity to input scaling. The thesis also investigates the trade-offs between accuracy and interpretability, as well as the current challenges and research gaps in Symbolic Regression for surrogate modeling.

TODO: Add Results and Conclusion to Abstract.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation

Scientific and engineering research mostly relies on numerical simulations and physical experiments to investigate complex systems, optimize technical designs, and evaluate control strategies. High-fidelity simulations may require expensive numerical solvers, while physical experiments may be time consuming and costly. These costs become a practical bottleneck when a system must be evaluated repeatedly, for example during optimization, uncertainty quantification, sensitivity analysis, or parameter exploration [2].

A surrogate is trained on observations obtained from simulations, physical measurements, or both and is then used to provide inexpensive predictions for new inputs. This approach is widely used to manage computational complexity in engineering design and simulation-based analysis, where the cost of generating training data must be balanced against predictive accuracy and the computational savings achieved during repeated evaluation [2]. Surrogate models are particularly useful in domains such as physics, mechanical engineering, materials science and energy systems, where obtaining direct simulation results or experimental measurements can be expensive.

Modern machine learning methods are common choices for surrogate modeling because they can approximate complex nonlinear relationships. Neural networks, ensemble models, Gaussian processes, and other flexible approaches achieve strong predictive performance [2]. However, this flexibility can make the resulting surrogate difficult to inspect, explain, or compare with known scientific relationships. This lack of transparency may be acceptable when a surrogate is used only for prediction. However, when the model is intended to support scientific understanding, engineering decisions, or safety-relevant applications, predictive accuracy alone may be insufficient, and the structure and behavior of the learned model must also be considered.

1.2 The Need for Interpretable Models

Interpretability becomes important when a model is intended to support scientific understanding rather than prediction alone. In scientific and engineering applica-

tions, an interpretable surrogate can help researchers inspect which variables appear in the learned relationship, examine their nonlinear interactions, and compare the model structure with established domain knowledge. These forms of inspection are difficult when the surrogate is a black box since a low prediction error on testing data does not by itself demonstrate that a model has learned a scientifically meaningful or domain-consistent relationship.

The distinction between interpretable models and post-hoc explanations is important. Rudin argues that in high-stakes settings, one should prefer models that are interpretable by design instead of first training a black-box model and then trying to explain it afterward [17]. This argument is also relevant for scientific surrogate modeling. If the learned model is meant to support reasoning about a physical or engineering system, the transparency of the model itself is important.

Interpretability is not a single property. A model may be considered interpretable because it is simple, because it uses familiar mathematical operations or because its variables have clear roles. In surrogate modeling, interpretability also must be balanced against accuracy. A very simple model may be easy to understand but too inaccurate to be useful, while a highly accurate model may be too complex to provide valuable insight. This trade-off motivates benchmarking the different frameworks according to both predictive accuracy and model complexity [13, 9].

1.3 Symbolic Regression as a Candidate Approach

Symbolic Regression (SR) is a machine learning approach that searches for explicit mathematical expressions fitting a given dataset. Instead of assuming a fixed model form, such as a linear model or a predefined polynomial basis, SR explores combinations of variables, constants, and mathematical operators. The result is not only a predictor but also an equation that can be read, evaluated, simplified, and compared with domain knowledge [14, 10].

This makes SR a promising candidate for interpretable surrogate modeling. A symbolic expression can often be much cheaper to evaluate than the original simulation and more transparent than the black-box models. SR has therefore received considerable attention in scientific discovery, including physics, materials science, and dynamical systems [3, 12, 22, 4].

Several well-known examples illustrate the scientific motivation for SR. AI Feynman uses physics-inspired strategies to recover equations from the Feynman Lectures on Physics, showing how prior structural ideas such as symmetry and separability can support symbolic equation discovery [21]. Sparse Identification of Nonlinear Dynamics (SINDy) similarly pursues interpretable governing equations from data, although it uses sparse regression over a predefined library of candidate terms rather

than general symbolic search [5]. These works show the broader appeal of data-driven equation discovery for scientific modeling.

At the same time, SR is not automatically superior to other surrogate modeling approaches. The search space of possible expressions is large, and different frameworks use different strategies to explore it. Evolutionary algorithms, genetic programming variants, and neural or reinforcement-learning-based approaches differ in their assumptions, computational cost, ease of use, and ability to discover compact expressions [10, 16]. The quality of the final model also depends on operator choices, complexity penalties, training budgets, and implementation details.

1.4 Research Question and Contributions

The central research question of this thesis is:

> *How suitable are Symbolic Regression frameworks for interpretable surrogate modeling, and how do they compare with respect to predictive performance, computational cost, equation complexity, and practical usability?*

To answer this question, this thesis conducts a systematic study of six open-source SR frameworks: gplearn [20], Operon [6], PySR [7], GeneticEngine [11], ITEA [8], and EQL [18]. The selected frameworks represent different approaches to symbolic model discovery, including tree-based genetic programming, efficient evolutionary search, grammar-guided genetic programming, interaction transformation based evolutionary search, and neural equation learning. The frameworks are evaluated on eight analytical, simulated, and empirical surrogate-modeling problems using fixed training and test datasets and documented algorithm-specific search budgets. The comparison considers predictive accuracy, computational efficiency, expression complexity, recovery of known equations, stability across repeated trials, sensitivity to input scaling, and practical reproducibility.

The main contributions of this thesis are:

- a structured overview of Symbolic Regression as an approach for interpretable surrogate modeling;
- a benchmark setup for comparing the selected SR frameworks on datasets from different application domains;
- an evaluation protocol covering predictive accuracy, runtime, setup effort, equation complexity, and stability in repeated prediction settings;
- an interpretability-oriented comparison of the discovered symbolic expressions;
- practical guidance on strengths, limitations, and challenges of the current SR frameworks for surrogate modeling.

The remainder of this thesis is organized as follows. Chapter 2 introduces the background on surrogate modeling, Symbolic Regression, method families, and interpretability. Chapter 3 discusses related work, including existing reviews, benchmark studies, and applications. Chapter 4 describes the methodology, including framework and dataset selection, evaluation metrics, reproducibility setup, and interpretability. Chapter 5 presents the experimental setup. Chapter 6 reports the results. Chapter 7 discusses the findings and their limitations. Chapter 8 concludes the thesis and outlines directions for future work.

2 Background

2.1 Surrogate Modeling

Many scientific and engineering workflows rely on simulation models that provide detailed representations of physical systems but are computationally expensive to evaluate. Finite element analysis and computational fluid dynamics are prominent examples. Their computational cost becomes a practical limitation when numerous evaluations are required for tasks such as design optimization, uncertainty quantification, sensitivity analysis, or exploration of a parameter space [2].

A surrogate model approximates the input output relationship of the computationally expensive simulation or physical process. It is constructed from a finite set of observations and is intended to provide predictions at a substantially lower evaluation cost than the original process [2]. If the original model is denoted by

$$y = f(\mathbf{x}), \tag{2.1}$$

where $\mathbf{x} \in \mathbb{R}^d$ is an input vector and $y \in \mathbb{R}$ is the corresponding output, a surrogate model approximates this relationship with a function $\hat{f}$ such that

$$\hat{y} = \hat{f}(\mathbf{x}) \approx f(\mathbf{x}). \tag{2.2}$$

The main practical advantage of a surrogate is that evaluating $\hat{f}$ is generally less computationally expensive than evaluating the original model f. This makes repeated model evaluations feasible, but computational efficiency alone does not determine whether a surrogate is useful. Its predictive accuracy on relevant unseen data, the amount of data required for its construction, and the computational cost of fitting and evaluating it must also be considered [2].

A wide range of statistical and machine-learning methods can be employed as surrogate models, including polynomial response surfaces, Gaussian processes, artificial neural networks, support vector regression, boosted trees, and random forests [2]. These methods can approximate complex nonlinear relationships without requiring a complete analytical description of the underlying process. However, flexible predictive models may provide limited insight into the learned input output relationship [14].

2.2 Symbolic Regression

Symbolic Regression (SR) is a supervised learning approach that searches for mathematical expressions fitting a given dataset. In contrast to ordinary regression methods, SR does not assume a fixed functional form in advance. For example, a linear regression model assumes that the target can be represented as a weighted sum of input variables. A polynomial regression model assumes a predefined set of polynomial terms. SR instead searches over both the structure of the expression and its numerical parameters [3, 13].

Symbolic regression (SR) searches for a mathematical expression that describes the relationship between input variables and a target variable directly from data [14]. Consider a supervised dataset

$$D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \tag{2.3}$$

where D denotes the complete dataset, N is the number of observations, and $i \in \{1, \ldots, N\}$ indexes an individual observation. The vector $\mathbf{x}_i \in \mathbb{R}^d$ contains the values of the d input variables for the i-th observation, while $y_i \in \mathbb{R}$ is its corresponding target value.

The objective of SR is to find both a symbolic expression structure ϕ and numerical constants $\boldsymbol{\theta}$ such that the prediction $\phi(\mathbf{x}_i; \boldsymbol{\theta})$ approximates y_i. For example, one possible measure of prediction error is the mean squared error (MSE),

$$\mathcal{L}_{\text{MSE}}(\phi, \boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \phi(\mathbf{x}_i; \boldsymbol{\theta})\right)^2 . \tag{2.4}$$

where $y_i - \phi(\mathbf{x}_i; \boldsymbol{\theta})$ denotes the prediction error for the i-th observation.

However, minimizing prediction error alone can produce unnecessarily large expressions. Because the result of SR is intended to remain interpretable, expression complexity must also be considered. Complexity may, for example, be measured by the number of nodes or operators in the expression tree. SR is therefore commonly treated as a multi objective problem involving a trade-off between predictive accuracy and expression complexity [7, 13].

Candidate expressions are commonly represented as syntax trees. In such a tree, internal nodes are mathematical operators such as $+$, $-$, $\times$, division, sin, cos, log, or exp, while leaf nodes are variables or constants. For example, the expression

$$\phi(x_1, x_2) = 3x_1 + \sin(x_2/5) \tag{2.5}$$

can be represented as a tree combining multiplication, division, addition, two constants, two variables, and a sine operator.

TODO: Add an actual tree diagram.

This representation makes it possible to modify expressions by changing subtrees, replacing operators, or adjusting constants.

The search space of SR is very large. Even a small set of variables and operators can generate many possible expressions, and the number of possible expressions grows quickly with expression size. This is one reason why SR is computationally difficult and why many frameworks use different search strategies [10]. Another challenge is the treatment of constants. In scientific equations, real-valued constants are often essential, but searching simultaneously over expression structure and continuous constants is much harder than fitting coefficients in a fixed model form [7].

Despite these challenges, SR is useful for scientific modeling because its outputs can be interpreted directly. A discovered equation can be evaluated cheaply, simplified symbolically, compared with existing theory, inspected for asymptotic behavior, and checked for physically implausible terms. This distinguishes SR from many black-box methods, where interpretability is usually added after training through explanation methods rather than being part of the model itself [14].

2.3 Main Symbolic Regression Method Families

Recent surveys group SR methods into several broad families. The most relevant distinction for this thesis is between deterministic methods, genetic programming methods, and neural methods [10]. These families differ in how they explore the expression space and handle constants.

Deterministic methods use systematic search, enumeration, sparse regression, mixed-integer optimization, or related strategies to explore candidate expressions. Their main advantage is that they can be more stable and reproducible than purely stochastic methods, because the search process is less dependent on random initialization. Some deterministic approaches can also provide stronger guarantees within a restricted search space. The limitation is that the number of possible expressions grows rapidly with the number of variables, operators, and expression depth. As a result, deterministic methods may become computationally expensive for high-dimensional or complex problems [10].

Genetic Programming (GP) methods are historically the most established family of SR algorithms. GP treats expressions as individuals in a population and evolves them over many generations. Candidate expressions are evaluated according to a fitness function, selected, recombined, mutated, and simplified. This family is flexible because it can search over many different expression structures without

requiring a fixed basis. It also naturally supports multi-objective optimization, where accuracy and complexity are optimized together [3, 13].

TODO: Maybe swap DSR example for EQL.

Neural-network-based SR methods use neural networks to guide or perform the expression search. Instead of evolving expressions only through classical GP operators, these methods may train a recurrent neural network, transformer, or other neural architecture to generate candidate formulas. Deep Symbolic Regression (DSR), for example, uses reinforcement learning and a generative recurrent neural network to search for expressions [14, 13]. Neural approaches are motivated by the ability of deep learning systems to learn search heuristics from data or from many generated examples. Their limitations include training complexity, sensitivity to the distribution of training problems, and the risk that strong performance on synthetic benchmarks does not directly transfer to scientific data.

The variety of method families is important for this thesis because there is no single best SR approach. A framework that performs well on one type of problem may behave differently on datasets with noisy data or high-dimensional data. Therefore, the comparison in this thesis treats SR frameworks not only as implementations of one idea, but as tools with different algorithmic assumptions.

2.4 The Accuracy Complexity Trade-Off

The central evaluation problem in SR is the trade-off between predictive accuracy and expression complexity. If only accuracy is optimized, SR can produce long expressions that fit the training data well but are difficult to interpret and prone to overfitting. If only simplicity is optimized, the resulting expression may be readable but too inaccurate to serve as a useful surrogate. A meaningful SR model should therefore achieve an appropriate balance between these objectives.

Complexity can be measured in different ways. A common syntactic measure is the number of nodes, operators, variables, and constants in an expression. Other measures consider expression depth, operator types, semantic behavior, nonlinearity, or the effective difficulty of understanding the formula [13, 10]. There is no perfect complexity metric. A short equation can still be difficult to interpret if it contains unfamiliar or unstable operations, while a slightly longer equation may be more meaningful if it matches known domain structure.

Many SR frameworks return not only one expression but a set of candidate expressions along a Pareto front. A Pareto front contains models for which no other model is both more accurate and less complex. This is useful because the final choice of model depends on the application. For a deployment-focused surrogate, a slightly more complex expression may be acceptable if it improves predictions significantly.

For scientific insight, a simpler expression may be preferable if it captures the dominant relation and remains physically plausible.

In this thesis, the accuracy complexity trade-off is treated as a core evaluation dimension. Predictive metrics such as mean squared error or coefficient of determination describe how well a model fits the data, while complexity metrics describe how complex the resulting expression is. Considering both dimensions is essential because a symbolic surrogate is only useful if it is accurate enough to approximate the underlying process and simple enough to support interpretation.

2.5 Interpretability in Scientific Modeling

Interpretability is a major motivation for applying SR to surrogate modeling. In scientific contexts, models are often expected to do more than predict. They should help explain a underlying system, reveal relationships between variables, and support reasoning about behavior in new conditions. A symbolic expression is attractive because the full mapping from inputs to outputs is visible in mathematical form.

The distinction between interpretability and post-hoc explainability is important. A black-box model may be explained after training using feature importance scores, saliency methods, local approximations, or other explanation techniques. These methods can be useful, but they do not make the original model transparent. An interpretable model, by contrast, exposes its structure directly. SR belongs to this second category, as it can produce sufficiently compact and meaningful expressions [14].

SR can support scientific discovery when the resulting expression reveals a relation that was not explicitly provided to the model. Reviews of SR in physical sciences emphasize this potential. SR can generate analytical equations from data and thereby connect data-driven modeling with the traditional scientific practice of formulating laws and empirical relations [3, 14]. At the same time, interpretability should not be assumed automatically. A symbolic expression may be mathematically explicit but still too complex, unstable, or domain-inconsistent to be useful.

This thesis treats interpretability as an evaluation criterion rather than as a guaranteed outcome. The discovered expressions will be compared in terms of their size, structure, operator usage, and practical readability. This allows SR frameworks to be evaluated not only by whether they predict accurately, but also by whether they produce surrogate models that can plausibly support scientific understanding.

3 Related Work

3.1 Symbolic Regression Reviews

The recent growth of Symbolic Regression (SR) has led to several survey papers that organize the field and summarize its main technical directions. Makke and Chawla provide a broad review of SR as a method for interpretable scientific discovery, emphasizing the ability of SR to infer compact mathematical expressions directly from data [14]. Their review discusses the historical development of SR, its relation to genetic programming, the emergence of deep-learning-based approaches, and applications in scientific domains. This perspective is closely aligned with the motivation of this thesis, because surrogate modeling in science and engineering often requires models that are not only accurate but also interpretable.

Angelis et al. focuses specifically on SR in the physical sciences [3]. Their review positions SR as a response to the limitations of black-box machine learning models in physics-based computation. They discuss SR as a data-driven method that can generate analytical equations and thereby support interpretation, generalization, and the discovery or verification of physical relations.

Dong and Zhong provide a more recent and comprehensive survey of advances in SR [10]. They organize SR methods into deterministic, genetic-programming-based, and neural-network-based families, and they also summarize benchmark datasets, evaluation metrics, software tools, and open research challenges. Their survey is useful for understanding the current methodological landscape and for motivating the framework selection in this thesis. In particular, the distinction between GP-based and neural SR approaches is relevant because the evaluated frameworks represent different search and implementation strategies.

These reviews show that SR is no longer limited to genetic programming. Current SR research includes evolutionary algorithms, deterministic search, sparse regression, reinforcement learning, transformer-based approaches, and hybrid methods. At the same time, the reviews also point to persistent challenges. For example, high computational cost, sensitivity to benchmark design, difficulty in evaluating interpretability, and inconsistent behavior across datasets.

3.2 Benchmark Studies

Benchmarking is a central issue in SR research because methods are often evaluated on different datasets, with different computational budgets, and with different definitions of success. Early SR papers frequently used small synthetic benchmarks, which made results difficult to compare across studies. Orzechowski, La Cava, and Moore addressed this issue by evaluating recent SR methods against standard machine learning regressors on a large set of regression problems [15]. Their study compared four GP-based SR methods with several scikit-learn regression methods on 94 real-world datasets. The results showed that SR methods can be competitive with strong machine learning baselines, but also that runtime remains as a limitation.

La Cava et al. extended this line of work with SRBench, a larger and more systematic benchmark for symbolic regression [13]. SRBench compares 14 SR methods and 7 machine learning methods across 252 regression problems, including real-world datasets, ground-truth symbolic equations, physics equations, and systems of ordinary differential equations. Importantly, SRBench evaluates not only predictive accuracy but also expression complexity and symbolic solution recovery. The benchmark therefore provides a basis for comparing SR methods in terms of both predictive performance and expression complexity.

The SRBench results also highlight that no single method is universally optimal. Some frameworks perform strongly on black-box regression tasks, while others are better at recovering exact equations under controlled conditions. Operon, for example, is reported as one of the strongest methods on real-world black-box regression problems and as part of the best accuracy-simplicity trade-offs [13]. These findings motivate a more focused comparison of selected frameworks in the surrogate modeling context, where practical usability, and interpretability are important.

More recent work has extended and criticized existing benchmark practices. SRBench++ argues that expression size alone is an insufficient proxy for interpretability and introduces domain-expert interpretation into the benchmarking process [9]. It also evaluates SR algorithms on more specific task properties, such as feature selection, robustness, extrapolation, and local minima. This is important because a symbolic expression can be short but still scientifically unhelpful, or slightly longer but more meaningful to a domain expert.

Aldeia et al. present a call for the next generation of SR benchmarks, arguing that SRBench should evolve as a living benchmark [1]. Their work expands the set of evaluated methods, refines evaluation metrics, and considers trade-offs between accuracy, complexity, and energy consumption. The study emphasizes standardization of hyperparameter tuning, execution constraints, computational resources, and benchmark maintenance. These issues are directly relevant for this thesis because a fair comparison of SR frameworks requires a reproducible protocol.

Overall, benchmark studies show that SR evaluation is more complex than ordinary regression benchmarking. Accuracy, model complexity, solution recovery, runtime, robustness, and interpretability all matter. The present thesis builds on these insights but uses a narrower experimental scope. Instead of ranking many SR methods across hundreds of datasets, it compares a selected set of open-source frameworks under a reproducible surrogate modeling protocol.

3.3 Selected Framework Publications

This thesis evaluates six open-source Symbolic Regression frameworks: gplearn, Operon, PySR, GeneticEngine, ITEA, and EQL. They were selected to represent different approaches to symbolic model construction, including conventional tree-based genetic programming, performance-oriented evolutionary search, grammar-guided genetic programming, interaction transformation expressions, and neural equation learning. The associated publications describe the algorithms and software architectures, while the exact versions and configurations used in this thesis are specified separately in the experimental methodology.

gplearn

Gplearn is a Python implementation of conventional tree-based genetic programming with an interface modeled after scikit-learn [20]. Candidate expressions are represented as programs and evolved using tournament selection, crossover, and several forms of mutation, while a parsimony coefficient can be used to discourage unnecessarily large expressions. Unlike the other selected frameworks, gplearn does not have a dedicated peer-reviewed framework publication. However, it is included and characterized as a Koza-style symbolic-regression implementation in the SRBench comparison [13]. Its comparatively direct implementation and familiar estimator interface make it a useful reference point for evaluating more specialized SR approaches.

Operon

Operon is an efficient C++ genetic programming framework designed specifically for Symbolic Regression [6]. It represents expression trees as contiguous linear sequences, which improves memory locality and enables efficient evaluation and manipulation of candidate models. Its modular evolutionary design is organized around offspring generators and supports automatic differentiation and gradient-based optimization of numerical constants. Operon emphasizes runtime performance, memory efficiency, and extensibility, making it relevant for examining how an optimized genetic programming implementation performs within the benchmark.

PySR

PySR is an open-source library for scientific Symbolic Regression that provides a Python interface to the Julia backend SymbolicRegression.jl [7]. Its search procedure uses multiple evolving populations and an evolve simplify optimize loop in which candidate expressions are evolved, simplified, and refined through numerical constant optimization. PySR also supports user defined operators, losses, complexity measures, and expression export to several scientific-computing formats. These capabilities make it a flexible framework to search for symbolic models under application specific mathematical constraints.

GeneticEngine

GeneticEngine is a Python framework for grammar-guided genetic programming in which the search space can be defined using Python data types rather than a separate textual grammar [11]. The framework combines ideas from grammar-guided and strongly typed genetic programming, allowing the structures and constraints of valid candidate programs to be expressed through the host language's type system. The publication emphasizes the ergonomics and expressive power of this representation and demonstrates this through an open source Python implementation.

ITEA

The Interaction Transformation Evolutionary Algorithm (ITEA) searches within a restricted representation in which models are formed as linear combinations of transformed interactions between input variables [8]. Each interaction is defined through powers of the input variables and a selected transformation function, while the corresponding linear coefficients are fitted numerically. Rather than using conventional subtree crossover, ITEA evolves a population through mutation operations that add, remove, or modify terms, interactions, and transformations. This structured representation distinguishes ITEA from general expression-tree methods and provides direct control over the maximum number of terms in a model.

EQL

Equation Learning (EQL) represents mathematical expressions through a differentiable neural network whose units implement elementary operations such as identity, trigonometric functions, and multiplication [18]. Alternating linear transformations and operation-specific units allow the network to construct nonlinear analytical relationships, while sparsity regularization encourages a small number of active connections and therefore more concise expressions. The architecture is optimized using gradient-based methods rather than an evolutionary population, making EQL

methodologically distinct from the other frameworks in the benchmark. The evaluated adapter implements the Equation Learner approach described in the associated literature.

The selected frameworks cover several parts of the contemporary SR landscape. gplearn, Operon, PySR, and GeneticEngine use different forms of genetic or evolutionary search, ITEA evolves expressions within the more restricted interaction transformation representation and EQL constructs equations through a differentiable neural architecture.

4 Methodology

4.1 Framework Selection Criteria

The purpose of the benchmark is not to identify a universally best Symbolic Regression (SR) algorithm, but to compare representative frameworks under conditions relevant to surrogate modeling. Framework selection was guided by five criteria: (i) the method is described in a scientific publication or authoritative software documentation, (ii) an open-source implementation is publicly available, (iii) the implementation returns an explicit mathematical expression rather than predictions alone, (iv) it can be integrated into the common SRBench-derived evaluation pipeline, and (v) it contributes a relevant algorithmic or representational approach to the comparison. Practical installability, compatibility with the evaluation interface, and reproducible execution were also considered because technical failures and unavailable model expressions affect the usefulness of a framework in practice. This reflects recommendations that SR benchmarks should consider reproducibility, complexity, and practical execution in addition to predictive accuracy [9, 1].

Based on these criteria, six frameworks were included: gplearn, Operon, PySR, GeneticEngine, ITEA, and EQL. Together, they provide a range of evolutionary, representation-based, and gradient-based approaches to Symbolic Regression. Their exact configurations and execution environments are described in Chapter 5.

4.2 Dataset Selection Criteria

The benchmark problems were selected to represent scientific and engineering surrogate-modeling tasks while remaining feasible for repeated Symbolic Regression searches. Each problem was required to have numerical input variables, a continuous target, clearly documented feature meanings and domains, and a reproducible data source or generating function. The available data also had to support disjoint training and test sets without categorical encoding, missing-value imputation, or extensive feature engineering. Documented input domains were additionally required so that the effect of fixed domain-based input scaling could be evaluated consistently.

The selection combines problems with known analytical equations and problems for which no privileged ground-truth expression is available. Cantilever, Borehole, Piston, and Wing Weight provide analytical generating equations and therefore support

both predictive evaluation and symbolic recovery analysis. Combined Cycle Power Plant, Naval Propulsion, Gas Turbine NOx Emissions, and Concrete Compressive Strength represent simulator-derived, measured, or experimental relationships for which evaluation is based on held-out predictive performance, expression complexity, runtime, and stability rather than exact equation recovery.

The final set of eight problems covers several engineering domains, between three and ten input variables, different physical scales, and mathematical relationships of varying structural difficulty. The analytical problems include multiplication, division, integer and fractional powers, logarithms, square roots, and trigonometric terms, while the non-analytical problems introduce measurement variation and relationships not defined by a known closed-form equation. This combination enables the frameworks to be assessed both as equation-recovery methods and as practical data-driven surrogate models. The exact problem definitions, sample sizes, data sources, and train test construction procedures are presented in Chapter 5.

4.3 Benchmark Protocol

All experiments use a common evaluation pipeline derived from SRBench [13, 19]. Each problem has a fixed training table and a separate test table that are generated or selected before the algorithm repetitions are executed. Dataset construction uses documented problem specific seeds, whereas the repetition seed controls the symbolic regression algorithm where supported. Consequently, every framework and repetition receives identical observations, preventing variation in dataset sampling from being confused with variation in algorithm behavior. SHA-256 hashes of the uncompressed training and test tables are stored in each result to make this invariant auditable.

Every algorithm problem combination is evaluated under two input-scaling conditions. In the `raw` condition, the stored inputs are supplied to the estimator in their original physical units. In the `domain_minmax` condition, each input x_j with documented lower and upper bounds l_j and u_j is mapped to $[-1, 1]$ according to

$$z_j = 2\frac{x_j - l_j}{u_j - l_j} - 1. \tag{4.1}$$

The transformation uses fixed problem-domain bounds rather than statistics estimated from the training data, ensuring that it is identical across frameworks and repetitions and does not introduce test-data leakage. Target values are not scaled and remain in their original units under both conditions. Expressions learned from normalized inputs are transformed back to the original physical variables during symbolic post processing.

The final repetition protocol uses the predefined labels 2, 3, 5, 7, 11, 13, 17, 19, 23, and 29. These labels are passed to each estimator that exposes a supported random-seed parameter. The ITEA implementation does not expose such a parameter, so its ten executions are reported as uncontrolled repetitions rather than seed-controlled trials. With six algorithms, eight problems, two input-scaling conditions, and ten repetitions, the complete benchmark contains 960 expected trials.

Each trial fits one model using the fixed training target and evaluates it exclusively on the corresponding held-out reference-test set. A successful result, an explicit execution failure, and an absent trial are treated as three separate states. Failure records retain the algorithm, problem, scaling condition, repetition label, and process exit status, while the summary tables report successful, failed, and missing trials separately. Failed or missing runs are therefore not silently excluded from the reported experiment coverage.

Each framework uses one explicitly documented configuration across the benchmark, no hyperparameter optimization is performed using the final test results. The configured search effort is expressed through algorithm-specific controls, such as population sizes, generations, evaluation limits, iterations, or time limits. These settings were selected to make repeated execution feasible and do not impose equal computational budgets across frameworks. Runtime comparisons therefore describe the evaluated configurations and must not be interpreted as either equal compute comparisons or estimates of each framework's best attainable performance.

Shared execution controls restrict benchmark containers to one thread. After fitting each model predicts the complete test set five times to obtain a more reliable estimate of prediction time. Model fitting, prediction, and expression extraction are timed within the running algorithm environment. Container startup, image retrieval, dataset generation, result serialization, and symbolic post-processing are excluded from the reported algorithm runtime.

Symbolic parsing, transformation to physical coordinates, simplification, complexity measurement, and comparison with available ground-truth equations are performed as separate post-processing steps. Each stage has an independent timeout and writes a versioned analysis record without modifying the original trial result. Separating expression analysis from model execution ensures that a parsing or simplification failure does not invalidate an otherwise successful training and prediction trial.

4.4 Evaluation Metrics

4.4.1 Predictive Performance

Predictive performance is evaluated exclusively on the held-out reference-test set. For target values y_i, predictions $\hat{y}_i$, the test-set mean $\bar{y}$, and n test observations, the

benchmark reports mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination (R^2):

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|y_i - \hat{y}_i\right|, \tag{4.2}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2}, \tag{4.3}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2}{\sum_{i=1}^{n}\left(y_i - \bar{y}\right)^2}. \tag{4.4}$$

MAE measures the average absolute prediction error, whereas RMSE assigns greater weight to large errors. Both remain in the target's original unit because target values are not scaled. The coefficient of determination measures performance relative to prediction by the test-set mean and may be negative when a model performs worse than that reference. Because the target ranges differ substantially between problems, raw MAE and RMSE values are interpreted within each problem rather than ranked directly across the complete dataset collection.

To provide a dimensionless error measure, the benchmark additionally reports range-normalized RMSE:

$$\mathrm{NRMSE}_{\mathrm{range}} = \frac{\mathrm{RMSE}}{\max_i(y_i) - \min_i(y_i)}. \tag{4.5}$$

The normalization uses the range of the reference-test target and allows prediction errors to be interpreted relative to the observed target variation. It does not modify the target values supplied during model fitting or prediction. The maximum absolute prediction error is also retained in each individual trial record as an additional indicator of the largest observed test-set deviation.

4.4.2 Computational Efficiency

Computational efficiency is evaluated using wall-clock measurements obtained with a monotonic high-resolution timer. The benchmark records the duration of the complete estimator fitting call and the first prediction over the reference-test set. Prediction of the full test set is repeated five times, from which the mean, median, and sample standard deviation are calculated. The median duration is also divided by the number of test observations and reported in microseconds per sample. Expression extraction is timed separately, while symbolic post-processing, container startup, image retrieval, dataset generation, and result serialization are excluded from the algorithm runtime.

4.4.3 Expression Complexity

Every returned expression is parsed with SymPy to obtain framework-independent structural measures. These include expression-tree node count, tree depth, operator count and operator set, constant count, and the number and identity of variables used. The same measures are calculated after symbolic simplification where possible. If simplification fails or reaches its time limit, the unsimplified expression measures remain available and are identified as a fallback rather than being silently combined with simplified values. The adapter-specific model-size value is retained for reference but is not used as the primary cross-framework complexity measure because its definition differs between adapters.

4.5 Reproducibility Setup

The benchmark is organized around separate components for dataset preparation, framework adapters, trial evaluation, symbolic analysis, and result aggregation. Versioned JSON files under `configs/` define the problem set, repetition labels, input-scaling conditions, shared execution controls, and algorithm-specific parameters. Runner scripts under `scripts/` execute individual trials or the complete experiment matrix. The resolved configuration, its cryptographic hash, and the parameters accepted by the estimator are recorded with each trial result.

Algorithm environments are isolated using the published SRBench container images. The image references are pinned by immutable SHA-256 digests in `containers/images.lock`, preventing a later image update from silently changing the execution environment. The host virtual environment is used for dataset preparation, experiment coordination, symbolic post-processing, aggregation, and automated tests. Shared execution controls restrict the algorithm containers to one thread, while supported estimators receive the configured repetition seed. Frameworks without a controllable seed parameter are identified explicitly in the result metadata.

Dataset origin is recorded independently of the algorithm runs. Analytical datasets are generated from documented equations, domains, and fixed generation seeds. External datasets retain their source URLs, identifiers, selection procedures, and source-file hashes. Every trial additionally records hashes of the exact training and test tables it received, making it possible to verify that all frameworks were evaluated on identical observations.

Trial results are stored under `results/<problem>/<benchmark>/<input-scaling>/<algorithm>/` as JSON files indexed by repetition label. Symbolic parsing, coordinate transformation, simplification, and ground-truth comparison write separate analysis sidecars so that post-processing does not modify the original trial record. Aggregate CSV files are generated from these stored artefacts and report successful, failed, and missing trials explicitly. This structure allows tables and statistical

summaries to be regenerated without fitting the models again.

4.6 Interpretability Evaluation

TODO: Describe interpretability evaluation.

5 Experiments

This chapter describes the concrete realization of the benchmark protocol introduced in the preceding methodology chapter. It defines the experimental matrix, documents how the individual datasets were constructed, records the configurations used for the six symbolic regression frameworks, and explains the execution and symbolic post-processing procedures. The chapter also reports the coverage of the completed experiment matrix, including trials that ended in recorded failures.

5.1 Experimental Matrix and Task Roles

The experimental matrix combines eight regression problems, six symbolic regression frameworks, two input-scaling conditions, and ten repetition labels. This produces

$$8 \times 6 \times 2 \times 10 = 960 \tag{5.1}$$

distinct trial coordinates. The evaluated frameworks are gplearn, Operon, PySR, GeneticEngine, ITEA, and EQL. Each framework is evaluated on both the original input variables and a domain-normalized representation in which every input is mapped to the interval $[-1, 1]$. Target values remain in their original units under both conditions.

The benchmark problems were selected to fulfill complementary experimental roles rather than to form a homogeneous dataset collection. Four analytical problems provide known generating equations and permit controlled sampling over documented domains. The remaining four problems are based on measured, experimental, or simulator-generated data and test approximation under relationships for which the benchmark does not assume an exact symbolic ground truth.

Table 5.1: Problems and their roles in the experimental matrix.

Problem	Data type	Inputs	Train/test	Experimental role
Cantilever	Analytical	5	400/4000	Rational power-law relationship and exact equation recovery
Borehole	Analytical	8	400/4000	Higher-dimensional rational relationship and exact equation recovery

Problem	Data type	Inputs	Train/test	Experimental role
Piston	Analytical	7	400/4000	Nonlinear relationship containing a nested square root
Combined Cycle Power Plant	Measured	4	400/4000	Low-dimensional industrial energy dataset
Naval Propulsion	Simulator-generated	3	400/4000	Propulsion-system surrogate with correlated operating variables
Wing Weight	Analytical	10	400/4000	Higher-dimensional expression with fractional powers and a trigonometric term
Gas Turbine NOx	Measured	9	400/4000	Industrial-emissions prediction with a chronological train–test separation
Concrete Strength	Experimental	8	400/592	Material-property prediction with replicated experimental observations

The analytical tasks emphasize controlled approximation and recovery of compact mathematical structure, whereas the empirical and simulator-generated tasks emphasize predictive performance on real-valued surrogate-modeling problems. Exact symbolic equivalence is evaluated automatically for Cantilever, Borehole, and Piston. Wing Weight also has a documented analytical generating equation, but its more complex combination of fractional powers and a trigonometric term is primarily used to evaluate approximation quality and expression structure in the present analysis.

5.2 Dataset Construction

5.2.1 Common Construction Procedure

All training and reference-test tables are constructed before the symbolic regression repetitions are executed. Dataset construction uses fixed problem-specific seeds that are independent of the framework repetition labels. The resulting tables are therefore identical for all frameworks and repetitions within a problem and input-scaling condition. SHA-256 hashes of the uncompressed tables are recorded in each trial result so that this property can be verified.

5.2.2 Analytical Datasets

The Cantilever problem is generated from its analytical rational power-law equation using five input variables. A single random generator initialized with seed 20260810 is used to draw the 400 training observations followed by the 4000 reference-test observations. The two tables are disjoint because they are produced by consecutive draws from the same fixed generator.

The Borehole problem contains eight input variables and is generated from the standard analytical borehole-flow equation. The training and reference-test designs are independently generated using scrambled Latin hypercube sampling with seeds 20260811 and 20260812, respectively. The equation contains several nonlinear interactions, including logarithmic and rational terms, making it a more demanding structural-recovery task than the Cantilever problem.

The Piston problem contains seven inputs and represents the cycle time of a simulated piston system. Scrambled Latin hypercube designs with seeds 20260813 and 20260814 are used for the training and reference-test tables. Its generating expression contains multiple intermediate interactions and a nested square root, allowing the benchmark to examine whether the frameworks can approximate a comparatively deep analytical structure.

The Wing Weight problem contains ten inputs and is generated from an analytical aircraft-wing weight equation. Independent scrambled Latin hypercube samples with seeds 20260817 and 20260818 produce the training and reference-test tables. The equation combines products, fractional powers, and a cosine term, thereby extending the analytical task collection beyond rational and square-root expressions.

5.2.3 External Datasets

The Combined Cycle Power Plant dataset is obtained from the UCI Machine Learning Repository and contains 9568 observations with four input variables. A fixed permutation generated with seed 20260815 is applied to the complete dataset. The first 400 permuted observations form the training set, and the following 4000 observations form the reference-test set. The source-file hash is pinned in the dataset specification to make the selected observations reproducible.

The Naval Propulsion problem is based on the UCI Condition Based Maintenance of Naval Propulsion Plants dataset. The source data contain a full factorial simulator design with 11934 observations. Three operating variables are used to predict the selected propulsion-system target. A permutation generated with seed 20260816 determines the 400 training and 4000 reference-test observations. Hashes of the downloaded archive and the extracted source member are retained with the dataset metadata.

The Gas Turbine NOx problem is constructed from five annual UCI gas-turbine emissions files containing 36733 observations in total. Nine ambient and process variables are used to predict nitrogen-oxide emissions, while the year indicator and carbon-monoxide target are excluded. Observations from 2011–2013 define the training pool, and observations from 2014–2015 define the reference-test pool. Seed 20260819 is used to sample 400 and 4000 observations without replacement from the respective pools. This chronological separation reduces direct overlap between the operating periods represented during fitting and evaluation.

The Concrete Strength problem is obtained from the UCI Concrete Compressive Strength dataset. Observations with identical values for all eight input variables are aggregated into distinct experimental designs, and their target values are combined, producing 992 unique designs. A fixed permutation generated with seed 20260820 assigns 400 designs to the training set and the remaining 592 designs to the reference-test set. The source-file hash and the aggregation procedure are recorded in the dataset specification.

5.2.4 Input Normalization

For the domain-normalized condition, each raw input x_j is transformed using the documented lower and upper bounds l_j and u_j of its problem domain:

$$z_j = 2\frac{x_j - l_j}{u_j - l_j} - 1. \tag{5.2}$$

These bounds are fixed by the dataset specification and are not estimated from either the training or reference-test observations. The transformation is therefore identical across frameworks and repetitions and does not introduce test-data leakage. Target values are not scaled.

5.3 Framework Configurations

All frameworks are executed with fixed configurations that are shared across problems, scaling conditions, and repetition labels. The configurations were selected before the final experiment matrix was executed, and no problem-specific hyperparameter optimization was performed. Table 5.2 summarizes the principal search controls.

Table 5.2: Principal framework configurations used in the experiments.

Framework	Search approach	Principal configuration
gplearn	Tree-based genetic programming	Population size 500 and 100 generations

Framework	Search approach	Principal configuration
Operon	Efficient tree-based genetic programming	Population size 1000, 1000 generations, and a maximum of 1,000,000 evaluations
PySR	Multi-population evolutionary search	20 iterations, 500 cycles per iteration, population size 100, three populations, and a 60-second timeout
GeneticEngine	Grammar-guided genetic programming	Population size 200, 100 generations, and a 60-second timer
ITEA	Interaction–transformation evolutionary search	Population size 1000 and 500 generations
EQL	Differentiable equation-learning network	10,000 training iterations

The gplearn, PySR, and GeneticEngine settings include project-level budget overrides introduced to keep the complete matrix computationally feasible. The Operon, ITEA, and EQL settings follow the parameter values exposed by their SRBench adapters. PySR is configured for deterministic serial execution, while the shared execution controls restrict all algorithm containers to one processing thread where supported.

The configurations do not represent equal wall-clock or equal-function-evaluation budgets because the frameworks expose different search controls and perform different internal operations. Runtime comparisons therefore describe the cost of the evaluated configurations rather than a hardware-normalized competition under exactly equivalent search effort.

The operator sets and estimator parameters accepted by each adapter are stored in the resolved trial records. This is important because similarly named operators may be implemented with different protected-domain behavior across frameworks. The recorded configuration and estimator parameters make these adapter-level differences visible instead of assuming that all frameworks search an identical expression space.

5.4 Execution Procedure and Environments

The experiment matrix is executed using the benchmark runner scripts, which select one problem, framework, input-scaling condition, and repetition label for each trial. The runner loads the fixed training and reference-test tables, applies the selected input representation, invokes the framework adapter, evaluates the returned estimator, extracts its symbolic expression, and writes the complete trial record as JSON. Failed fitting, prediction, or expression-extraction operations are recorded explicitly.

Each algorithm is executed in its published SRBench container environment. The six container references are pinned by immutable SHA-256 digests in `containers/images.lock`, preventing later image updates from silently changing the framework environments. Dataset preparation, experiment coordination, result validation, symbolic post-processing, and aggregation are performed in the project's host virtual environment.

Shared execution controls restrict algorithm containers to one thread to reduce uncontrolled parallelism between frameworks. The configured repetition label is passed to every estimator that exposes a compatible seed parameter. ITEA does not provide a controllable estimator seed through the employed adapter, and its repetitions are consequently marked as uncontrolled rather than being presented as deterministically reproducible seed runs.

For each successful fit, predictive metrics are computed on the complete reference-test set. The first prediction call is recorded separately, after which prediction over the same test table is repeated five times to estimate the mean, median, and sample standard deviation of prediction time. Expression extraction is timed independently. Container startup, image retrieval, dataset construction, symbolic post-processing, and result serialization are excluded from the reported algorithm runtime.

5.5 Repetitions and Trial Coverage

Each framework–problem–scaling combination is assigned the ten repetition labels

$$\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29\}. \tag{5.3}$$

These labels control the framework seed where supported, they do not alter the already constructed training and reference-test tables. Repetitions therefore measure variation in the symbolic regression procedure rather than variation caused by resampling the datasets.

All 960 coordinates of the planned matrix are represented by either a successful trial record or an explicit failure record. A total of 947 trials completed successfully and 13 trials failed during framework execution. Table 5.3 reports the resulting coverage by problem.

Table 5.3: Coverage of the completed experimental matrix.

Problem	Configuration suite	Planned	Successful	Failed
Cantilever	`v3`	120	120	0
Borehole	`v3`	120	120	0
Piston	`v3`	120	118	2
Combined Cycle Power Plant	`v4`	120	114	6

Problem	Configuration suite	Planned	Successful	Failed
Naval Propulsion	v5	120	120	0
Wing Weight	v6	120	117	3
Gas Turbine NOx	v7	120	118	2
Concrete Strength	v8	120	120	0
Total	–	960	947	13

All 13 recorded failures occurred under domain normalization. Twelve originated from GeneticEngine and one from Operon. These failures are retained as part of the benchmark outcome because numerical or adapter-level instability under a shared input transformation is relevant to practical framework comparison. They are excluded from metric averages but included when reporting completion and failure rates.

Statistical summaries are calculated from the available successful repetitions for each framework–problem–scaling combination. The number of successful trials is reported alongside every aggregate so that reduced coverage is visible. No metric values are imputed for failed trials, and out-of-protocol result files with repetition labels outside the configured set are ignored during aggregation.

5.6 Symbolic Post-Processing

Symbolic post-processing is performed after framework execution and writes a separate analysis sidecar for every successful trial. This separation preserves the original expression and metrics returned by the framework while allowing the symbolic analysis procedure to be repeated or improved without fitting the model again. Each sidecar records a hash of its source trial result so that stale analyses can be detected.

Expressions are translated into SymPy using parsers adapted to the expression formats returned by the frameworks. This includes conversion of the prefix-style function notation used by gplearn and the infix expressions returned by the other adapters. Parsing, coordinate transformation, simplification, and ground-truth comparison are treated as separate stages, each with a 60-second time limit. A failure or timeout in one symbolic stage is recorded explicitly and does not invalidate the predictive results of the corresponding trial.

For trials trained on normalized inputs, the symbolic variables z_j are replaced by the inverse of Equation 5.2,

$$z_j = \frac{2x_j - (u_j + l_j)}{u_j - l_j}, \tag{5.4}$$

to express the learned model in terms of the original physical variables. Candidate expressions are then normalized and simplified using operations such as cancellation

and factorization. The smallest valid candidate produced by the simplification procedure is retained as the principal simplified expression, while the original parsed expression remains available in the sidecar.

Framework-independent structural measures are computed before and after simplification. These measures include expression-tree node count, tree depth, operator count and operator set, constant count, and the number and identity of variables used. Complexity values associated with transformed expressions describe the equation in the original input coordinates, whereas the untransformed values document the structure directly returned by the estimator.

For Cantilever, Borehole, and Piston, the transformed expression is additionally compared with the documented generating equation. Exact equivalence is accepted only when symbolic simplification reduces the difference between the discovered and reference expressions to zero. Variable recovery and expression-size ratios are retained as supplementary structural measures because a numerically accurate model may recover only part of the generating structure or may express an equivalent relationship in a more complicated form.

Symbolic analysis sidecars are available for all 947 successful protocol-valid trials. Consequently, predictive, timing, and structural summaries can be generated from the same set of completed runs, while parsing or simplification fallbacks remain distinguishable from expressions that completed every symbolic analysis stage.

6 Results

6.1 Trial Coverage

Table 6.1: Coverage of the experimental matrix.

Problem	Planned	Successful	Failed
Cantilever	120	120	0
Borehole	120	120	0
Piston	120	118	2
Combined Cycle Power Plant	120	114	6
Naval Propulsion	120	120	0
Wing Weight	120	117	3
Gas Turbine NOx	120	118	2
Concrete Strength	120	120	0
Total	960	947	13

Table 6.2: Recorded failed trials.

Problem	Framework	Failed repetitions	Count
Piston	GeneticEngine	2, 13	2
Combined Cycle Power Plant	GeneticEngine	3, 5, 7, 13, 19	5
Combined Cycle Power Plant	Operon	7	1
Wing Weight	GeneticEngine	2, 3, 23	3
Gas Turbine NOx	GeneticEngine	3, 19	2
Total			13

6.2 Predictive Performance

6.2.1 Raw Inputs

Table 6.3: Mean range-normalized RMSE for the raw-input condition.

Problem	gplearn	Operon	PySR	GeneticEngine	ITEA	EQL
Cantilever	0.02004	3.70×10^{-8}	0.00885	0.03736	0.02693	0.02759
Borehole	0.08650	0.00095	0.08075	0.08871	0.01673	21.277
Piston	0.12170	0.01980	0.07466	0.06815	0.03774	2827.2
CCPP	0.17524	0.07685	0.07396	0.11389	0.06044	0.33061
Naval Propulsion	0.04301	0.01719	0.04369	0.06453	0.05473	0.19129
Wing Weight	0.08385	0.01197	0.09249	0.09389	0.01964	1.1258
Gas Turbine NOx	1.6051	0.18091	0.15364	0.19827	0.16260	1.0648
Concrete Strength	0.12577	0.09358	0.15076	0.12357	0.10729	0.40273
Median across problems	0.10410	0.01850	0.07770	0.09130	0.04623	0.73378

6.2.2 Domain-Normalized Inputs

Table 6.4: Mean range-normalized RMSE for the domain-normalized condition.

Problem	gplearn	Operon	PySR	GeneticEngine	ITEA	EQL
Cantilever	0.05087	0.01013	0.04239	0.09602	0.08066	0.02102
Borehole	0.12756	0.00837	0.06133	0.09728	0.07189	0.00715
Piston	0.16630	0.01990	0.05898	0.12117	0.06273	0.13550
CCPP	0.63118	0.07530	0.07291	0.20305	0.06479	0.10566
Naval Propulsion	0.05207	0.01703	0.02569	0.07498	0.02491	0.07684
Wing Weight	0.24328	0.01212	0.06828	0.13261	0.09237	0.00991
Gas Turbine NOx	0.23702	0.19524	0.15632	0.19976	0.15669	0.16693
Concrete Strength	0.13172	0.09002	0.17683	10499	0.15130	0.11628
Median across problems	0.14901	0.01847	0.06481	0.12689	0.07627	0.09125

6.3 Lowest Mean Error by Problem

Table 6.5: Framework with the lowest mean range-normalized RMSE for each problem and scaling condition.

	Raw inputs		Domain-normalized inputs	
Problem	Framework	NRMSE	Framework	NRMSE
Cantilever	Operon	3.70×10^{-8}	Operon	0.01013
Borehole	Operon	0.00095	EQL	0.00715
Piston	Operon	0.01980	Operon	0.01990
CCPP	ITEA	0.06044	ITEA	0.06479
Naval Propulsion	Operon	0.01719	Operon	0.01703
Wing Weight	Operon	0.01197	EQL	0.00991
Gas Turbine NOx	PySR	0.15364	PySR	0.15632
Concrete Strength	Operon	0.09358	Operon	0.09002

6.4 Runtime and Expression Complexity

Table 6.6: Median of the eight problem-level mean results for fitting time, prediction time, and simplified expression size.

Framework	Input condition	Fit time (s)	Prediction time (μs/sample)	Simplified nodes
gplearn	Raw	35.33	0.591	147.2
gplearn	Domain normalized	45.69	0.693	61.4
Operon	Raw	4.00	0.025	67.5
Operon	Domain normalized	3.86	0.025	66.0
PySR	Raw	89.86	0.709	9.1
PySR	Domain normalized	89.70	0.820	16.1
GeneticEngine	Raw	60.24	0.037	27.0
GeneticEngine	Domain normalized	60.24	0.042	29.7
ITEA	Raw	11.32	0.213	23.6
ITEA	Domain normalized	4.29	0.214	18.0
EQL	Raw	24.59	1.896	46.0
EQL	Domain normalized	24.44	1.895	64.7

6.5 Framework-Level Summary

Table 6.7: Framework-level results across both input conditions. NRMSE, R^2, expression size, and fitting time are medians of the 16 problem–scaling means.

Framework	Successful	Failed	Median NRMSE	Median R^2	Median nodes	Median fit time (s)
gplearn	160	0	0.12666	0.50638	108.4	36.61
Operon	159	1	0.01850	0.98892	66.4	3.89
PySR	160	0	0.07344	0.83956	9.8	89.80
GeneticEngine	148	12	0.10558	0.61006	27.5	60.24
ITEA	160	0	0.06376	0.88633	23.1	5.64
EQL	160	0	0.15122	−0.15328	50.4	24.51

6.6 Graphical Results

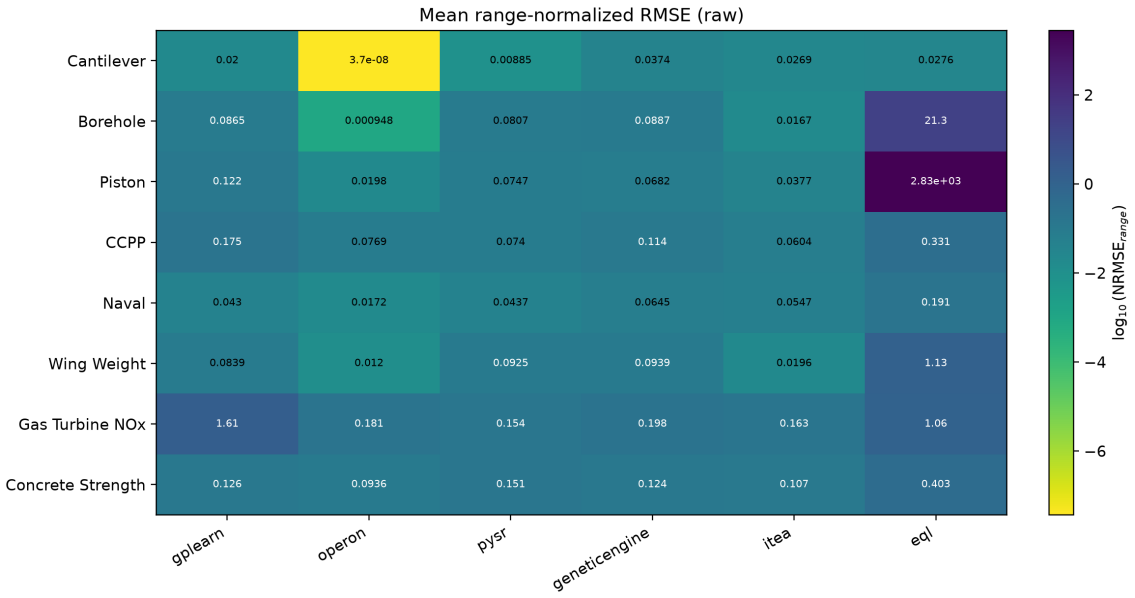

Figure 6.1: Mean range-normalized RMSE by problem and framework for the raw-input condition. Values are displayed on a logarithmic color scale.

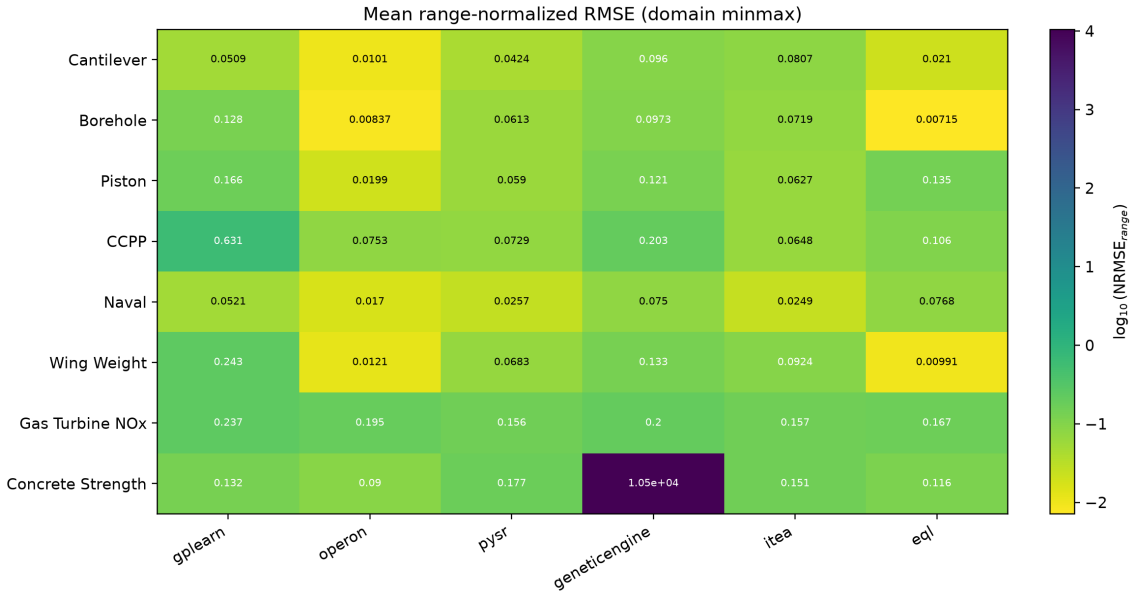

Figure 6.2: Mean range-normalized RMSE by problem and framework for the domain-normalized input condition. Values are displayed on a logarithmic color scale.

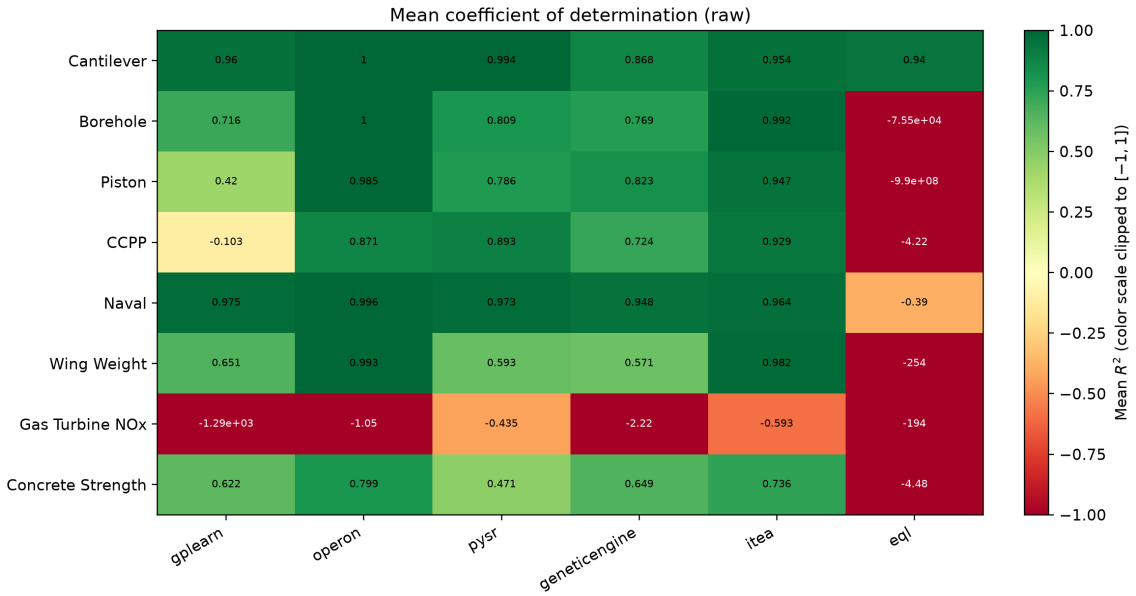

Figure 6.3: R^2 by problem and framework for the raw-input condition. Cell labels show the unmodified values, while the color scale is limited to the interval $[-1, 1]$.

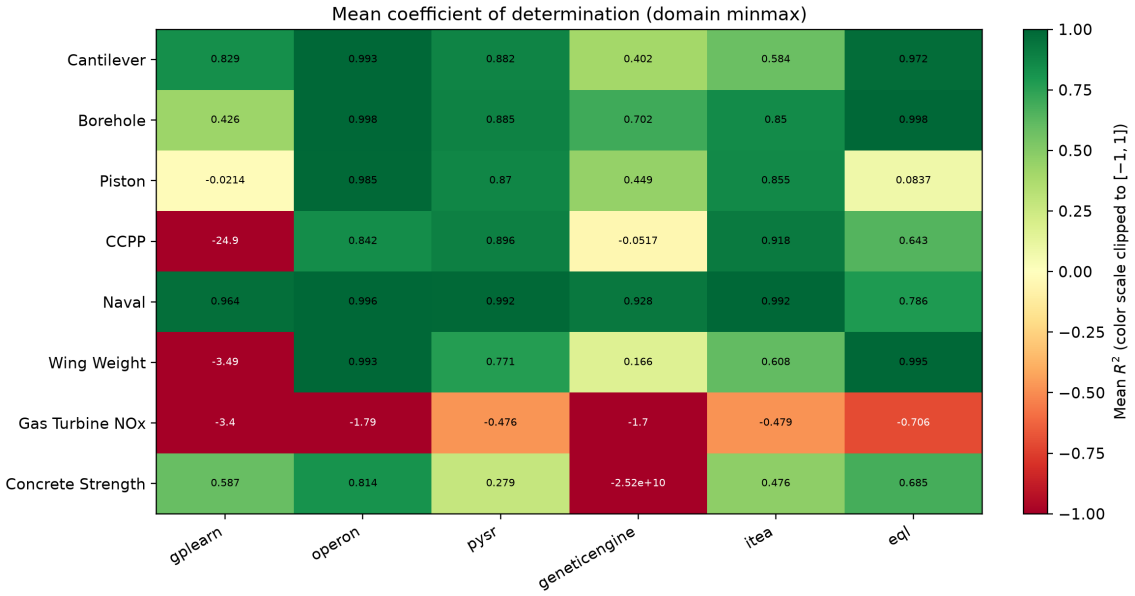

Figure 6.4: R^2 by problem and framework for the domain-normalized input condition. Cell labels show the unmodified values, while the color scale is limited to the interval $[-1, 1]$.

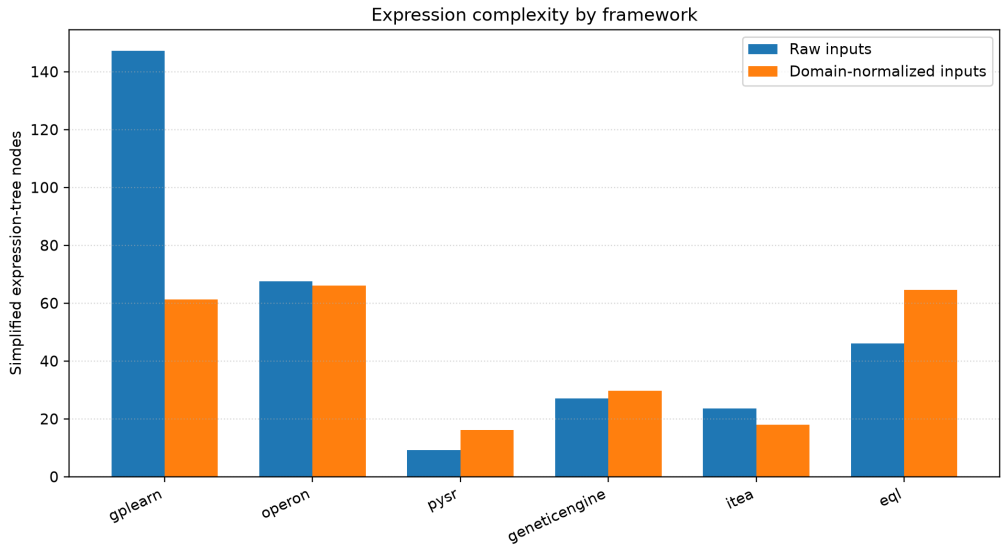

Figure 6.5: Median simplified expression-tree node count by framework and input-scaling condition. Values are medians of the eight problem-level means.

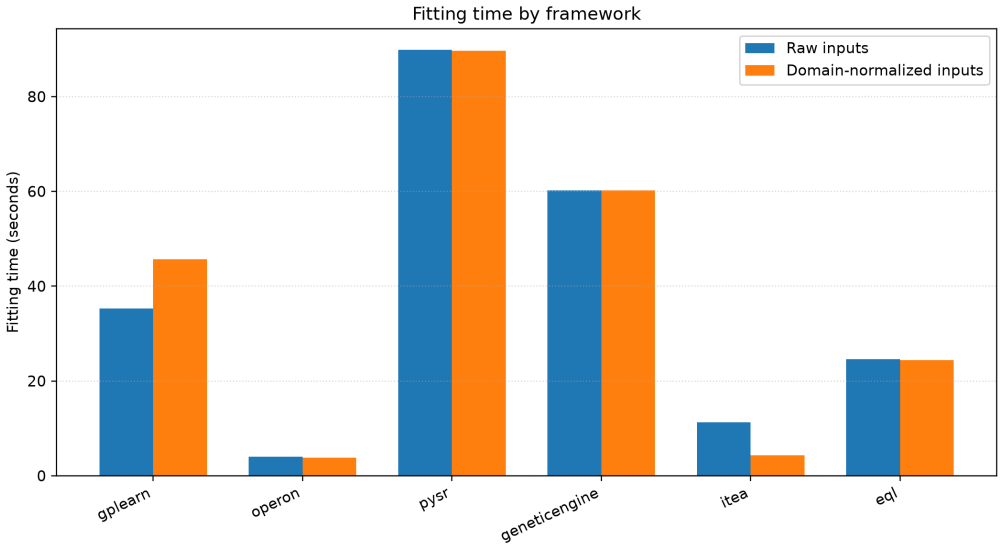

Figure 6.6: Median fitting time by framework and input-scaling condition. Values are medians of the eight problem-level means.

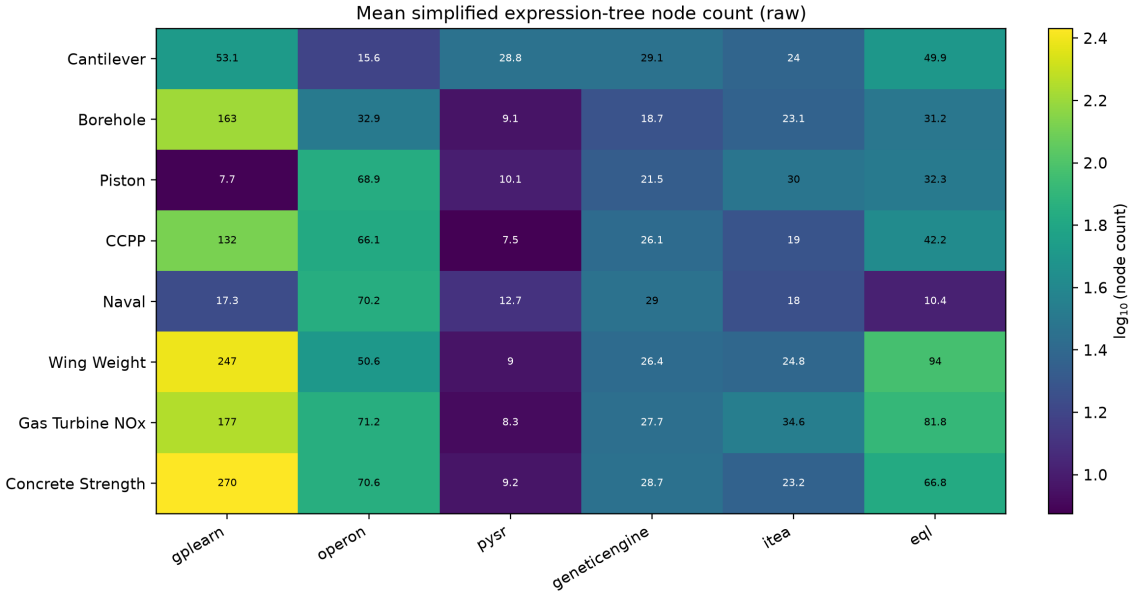

Figure 6.7: Mean simplified expression-tree node count by problem and framework for the raw-input condition. Values are displayed on a logarithmic color scale.

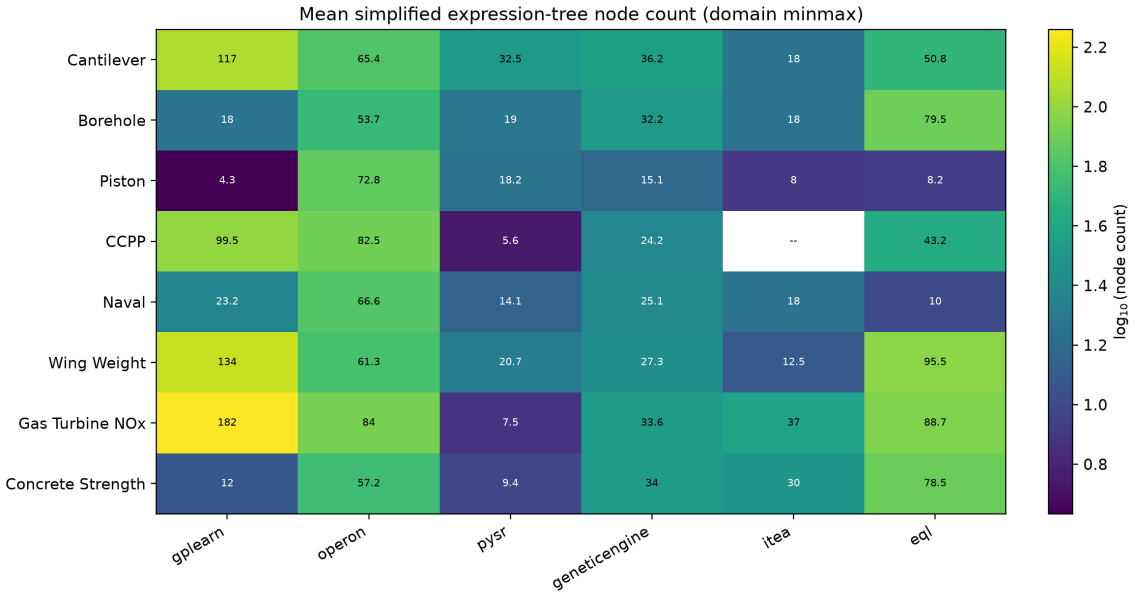

Figure 6.8: Mean simplified expression-tree node count by problem and framework for the domain-normalized input condition. Values are displayed on a logarithmic color scale.

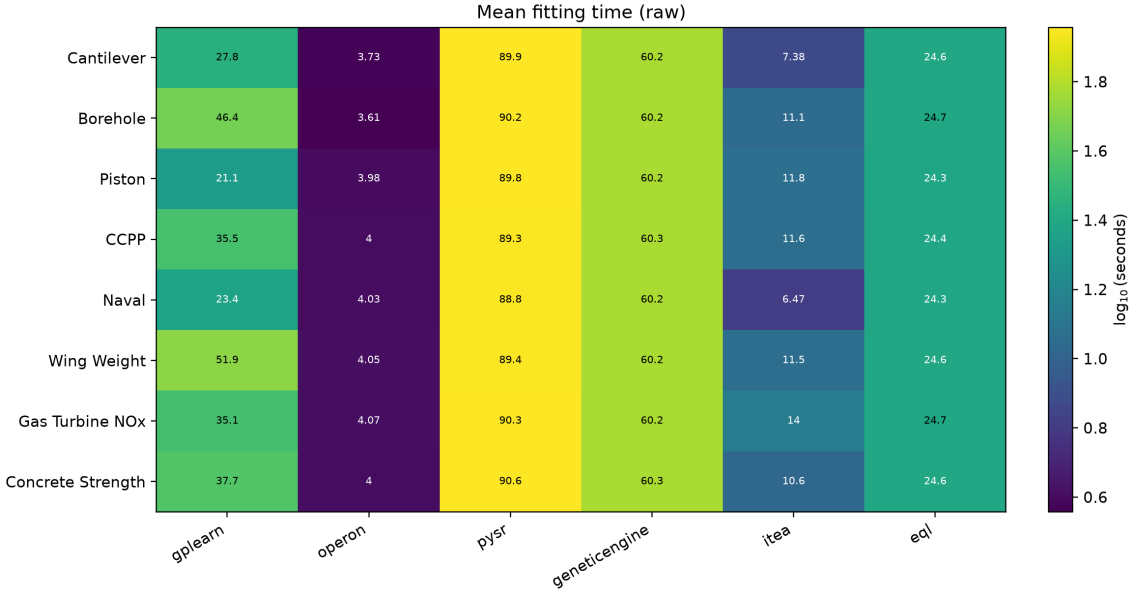

Figure 6.9: Mean fitting time by problem and framework for the raw-input condition. Values are shown in seconds on a logarithmic color scale.

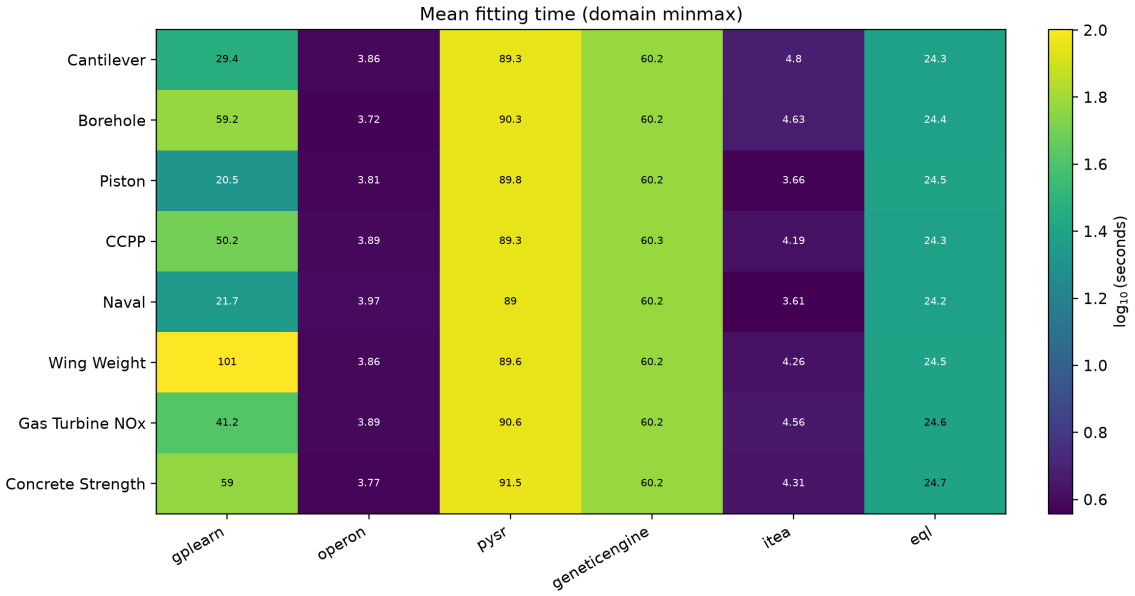

Figure 6.10: Mean fitting time by problem and framework for the domain-normalized input condition. Values are shown in seconds on a logarithmic color scale.

7 Discussion

7.1 Interpretation of Framework Trade-Offs

7.2 Suitability for Surrogate-Modelling Tasks

7.3 Practical Framework Selection

7.4 Limitations and Threats to Validity

7.5 Research Gaps and Future Benchmarking Needs

8 Conclusion

8.1 Summary of Findings

8.2 Answer to the Research Question

8.3 Future Work

Bibliography

[1] Aldeia, G. S. I., Zhang, H., Bomarito, G., Cranmer, M., Fonseca, A., Burlacu, B., La Cava, W. G. and de França, F. O. [2025], Call for action: Towards the next generation of symbolic regression benchmark, *in* 'Proceedings of the Genetic and Evolutionary Computation Conference Companion', Association for Computing Machinery, New York, NY, USA, pp. 1–10.
URL: *https://doi.org/10.1145/3712255.3734309*

[2] Alizadeh, R., Allen, J. K. and Mistree, F. [2020], 'Managing computational complexity using surrogate models: A critical review', *Research in Engineering Design* **31**(3), 275–298.
URL: *https://doi.org/10.1007/s00163-020-00336-7*

[3] Angelis, D., Sofos, F. and Karakasidis, T. E. [2023], 'Artificial intelligence in physical sciences: Symbolic regression trends and perspectives', *Archives of Computational Methods in Engineering* **30**(6), 3845–3865.
URL: *https://doi.org/10.1007/s11831-023-09922-z*

[4] Brum, B. R., Lober, L., Previdelli, I. and Rodrigues, F. A. [2026], 'Discovering equations from data: Symbolic regression in dynamical systems', *Journal of Physics: Complexity* **7**(1), 012001.
URL: *https://doi.org/10.1088/2632-072X/ae3eed*

[5] Brunton, S. L., Proctor, J. L. and Kutz, J. N. [2016], 'Discovering governing equations from data by sparse identification of nonlinear dynamical systems', *Proceedings of the National Academy of Sciences* **113**(15), 3932–3937.
URL: *https://doi.org/10.1073/pnas.1517384113*

[6] Burlacu, B., Kronberger, G. and Kommenda, M. [2020], Operon C++: An efficient genetic programming framework for symbolic regression, *in* 'Proceedings of the Genetic and Evolutionary Computation Conference Companion', Association for Computing Machinery, New York, NY, USA, pp. 1562–1570.
URL: *https://doi.org/10.1145/3377929.3398099*

[7] Cranmer, M. [2023], 'Interpretable machine learning for science with PySR and SymbolicRegression.jl'.
URL: *https://arxiv.org/abs/2305.01582*

[8] de França, F. O. and Aldeia, G. S. I. [2021], 'Interaction–transformation evolutionary algorithm for symbolic regression', *Evolutionary Computation*

29(3), 367–390.
URL: *https://doi.org/10.1162/evco_a_00285*

[9] de França, F. O., Virgolin, M., Kommenda, M., Majumder, M. S., Cranmer, M., Espada, G., Ingelse, L., Fonseca, A., Landajuela, M., Petersen, B. et al. [2025], 'SRBench++: Principled benchmarking of symbolic regression with domain-expert interpretation', *IEEE Transactions on Evolutionary Computation* **29**(4), 1127–1134.
URL: *https://doi.org/10.1109/TEVC.2024.3423681*

[10] Dong, J. and Zhong, J. [2025], 'Recent advances in symbolic regression', *ACM Computing Surveys* **57**(11), 1–37.
URL: *https://doi.org/10.1145/3735634*

[11] Espada, G., Ingelse, L., Canelas, P., Barbosa, P. and Fonseca, A. [2022], Data types as a more ergonomic frontend for grammar-guided genetic programming, *in* 'Proceedings of the 21st ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences', GPCE 2022, Association for Computing Machinery, pp. 86–94.
URL: *https://doi.org/10.1145/3564719.3568697*

[12] Guo, J. and Yin, W.-J. [2024], 'Harnessing data using symbolic regression methods for discovering novel paradigms in physics', *Science China Physics, Mechanics & Astronomy* **67**(6), 267301.
URL: *https://doi.org/10.1007/s11433-023-2346-2*

[13] La Cava, W. G., Orzechowski, P., Burlacu, B., de França, F. O., Virgolin, M., Jin, Y., Kommenda, M. and Moore, J. H. [2021], Contemporary symbolic regression methods and their relative performance, *in* 'Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks', Vol. 1.
URL: *https://openreview.net/forum?id=xVQMrDLyGst*

[14] Makke, N. and Chawla, S. [2024], 'Interpretable scientific discovery with symbolic regression: A review', *Artificial Intelligence Review* **57**(2).
URL: *https://doi.org/10.1007/s10462-023-10622-0*

[15] Orzechowski, P., La Cava, W. and Moore, J. H. [2018], Where are we now? a large benchmark study of recent symbolic regression methods, *in* 'Proceedings of the Genetic and Evolutionary Computation Conference', Association for Computing Machinery, New York, NY, USA, pp. 1183–1190.
URL: *https://doi.org/10.1145/3205455.3205539*

[16] Petersen, B. K., Landajuela Larma, M., Mundhenk, T. N., Santiago, C. P., Kim, S. K. and Kim, J. T. [2021], Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients, *in* 'International Conference on Learning Representations'.
URL: *https://arxiv.org/abs/1912.04871*

[17] Rudin, C. [2019], 'Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead', *Nature Machine Intelligence* **1**(5), 206–215.
URL: *https://doi.org/10.1038/s42256-019-0048-x*

[18] Sahoo, S. S., Lampert, C. H. and Martius, G. [2018], Learning equations for extrapolation and control, *in* 'Proceedings of the 35th International Conference on Machine Learning', Vol. 80 of *Proceedings of Machine Learning Research*, PMLR, pp. 4442–4450.
URL: *https://proceedings.mlr.press/v80/sahoo18a.html*

[19] SRBench contributors [n.d.], 'SRBench: A living benchmark framework for symbolic regression', GitHub repository. Revision ac1059bae0f641cf9e4dfd5c91add5d4e01dbcb5, accessed 21 August 2026.
URL: *https://github.com/cavalab/srbench*

[20] Stephens, T. [2026], 'gplearn: Genetic programming in python', Software documentation. Accessed 20 August 2026.
URL: *https://gplearn.readthedocs.io/en/stable/*

[21] Udrescu, S.-M. and Tegmark, M. [2020], 'AI Feynman: A physics-inspired method for symbolic regression', *Science Advances* **6**(16), eaay2631.
URL: *https://doi.org/10.1126/sciadv.aay2631*

[22] Wang, G., Wang, E., Li, Z., Zhou, J. and Sun, Z. [2024], 'Exploring the mathematic equations behind the materials science data using interpretable symbolic regression', *Interdisciplinary Materials* **3**(4), 637–657.
URL: *https://doi.org/10.1002/idm2.12180*